

UFUG 2602

Basic Sorting and Linked List

Tengfei Chang, Beiyan Cao

IoT Thrust and CES
Hong Kong University of Science and Technology (Guangzhou)

Outline

- Basic Sorting
 - **Bubble Sort**
 - Selection Sort
 - Counting Sort
- Linked List
 - Singly Linked List
 - Doubly Linked List
 - Link-Based Vs Array-based Sequences

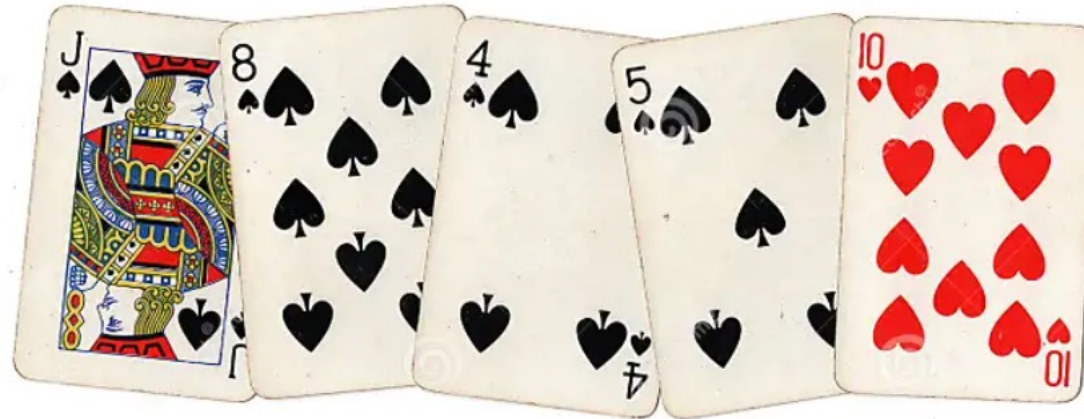
One Main Idea for Sorting

- Always consider the array has **two portions**: the **sorted portion** followed by the **unsorted portion**.
 - Initially, the sorted portion is empty, and the entire array is unsorted.
 - Enlarge the sorted portion by adding one more number in every iteration, and shrink the unsorted portion accordingly.
- Insertion sort is an instance of this idea. This lecture will introduce two more algorithms using this idea.

Bubble Sort – An example



- Imagine you're at a friendly card game night with your friends.
- Each of you has been dealt a hand of cards, and your task is to arrange your hand from the lowest card to the highest.
- However, you can only look at and **swap cards that are next to each other**.



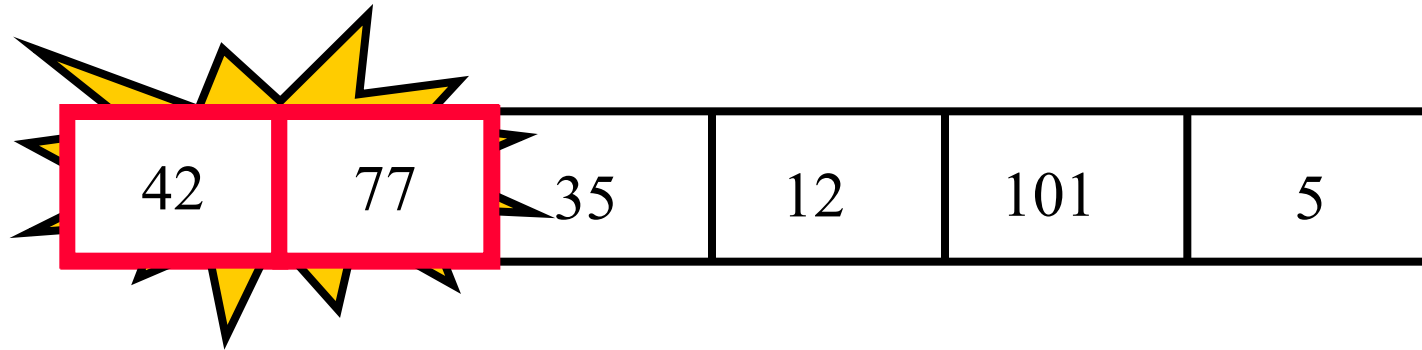
"Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**

77	42	35	12	101	5
----	----	----	----	-----	---

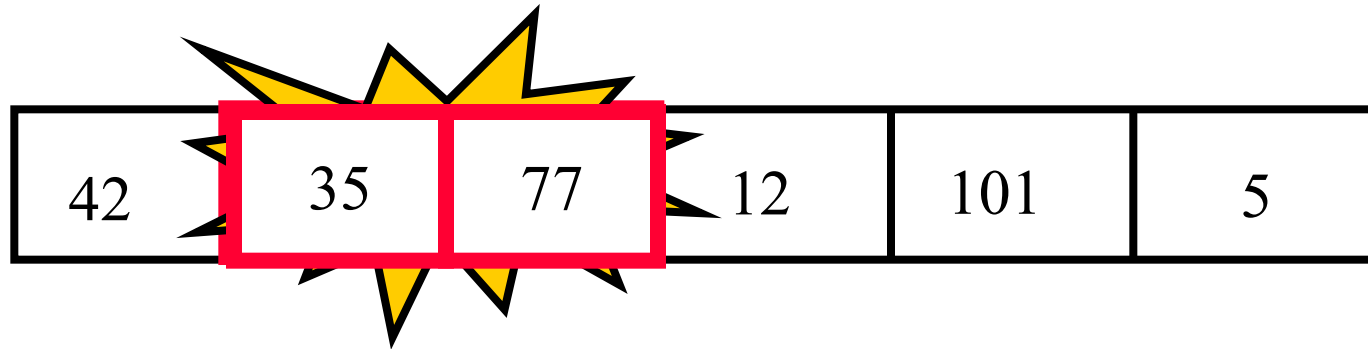
"Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**



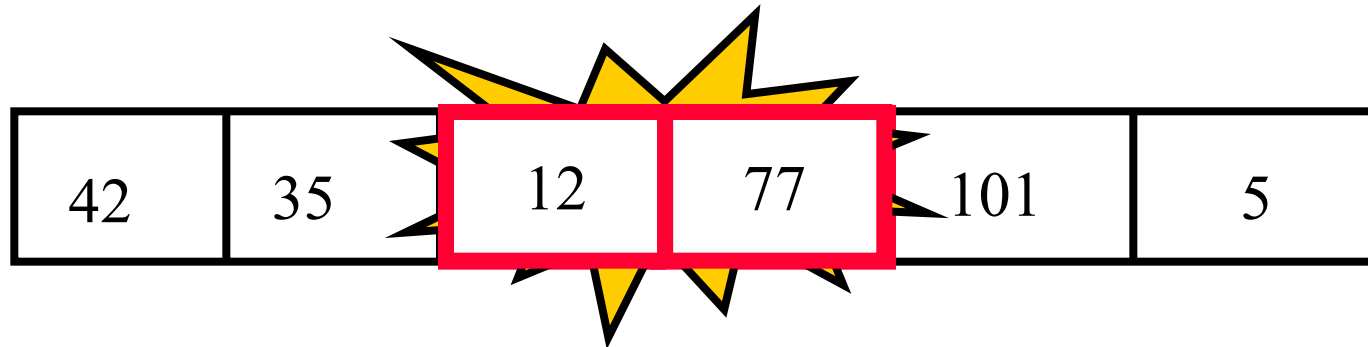
"Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- "Bubble" the **largest value** to the end using **pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

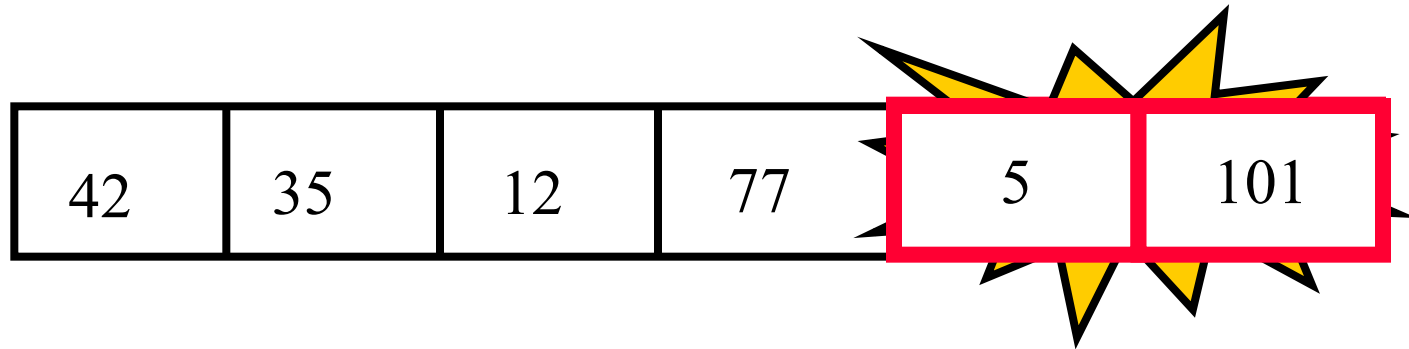
- Traverse a collection of elements
- Move from the front to the end
- "Bubble" the **largest value** to the end using **pair-wise comparisons and swapping**



No need to swap

"Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**



"Bubbling Up" the Largest Element

- Traverse a collection of elements
- Move from the front to the end
- "Bubble" the **largest value** to the end using **pair-wise comparisons and swapping**

42	35	12	77	5	101
----	----	----	----	---	-----

Largest value correctly placed

The “Bubble Up” Algorithm

Alg.: BUBBLE-UP(A)

for $j \leftarrow 1$ **to** $n - 1$

if $A[j] < A[j - 1]$

then exchange $A[j] \leftrightarrow A[j - 1]$

Items of Interest

- Notice that only the largest value is correctly placed
- All other values are still out of order
- So we need to repeat this process

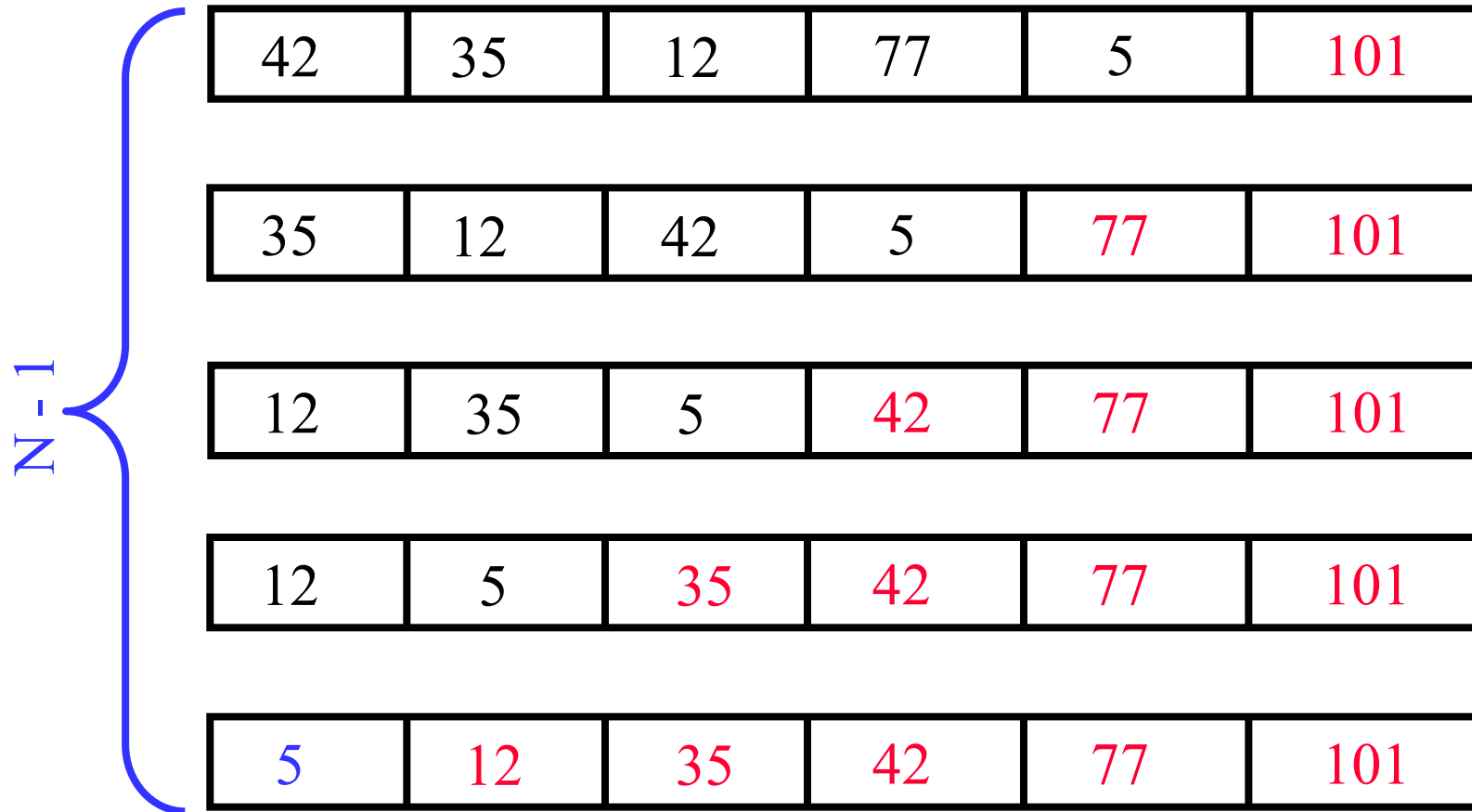
42	35	12	77	5	101
----	----	----	----	---	-----

Largest value correctly placed

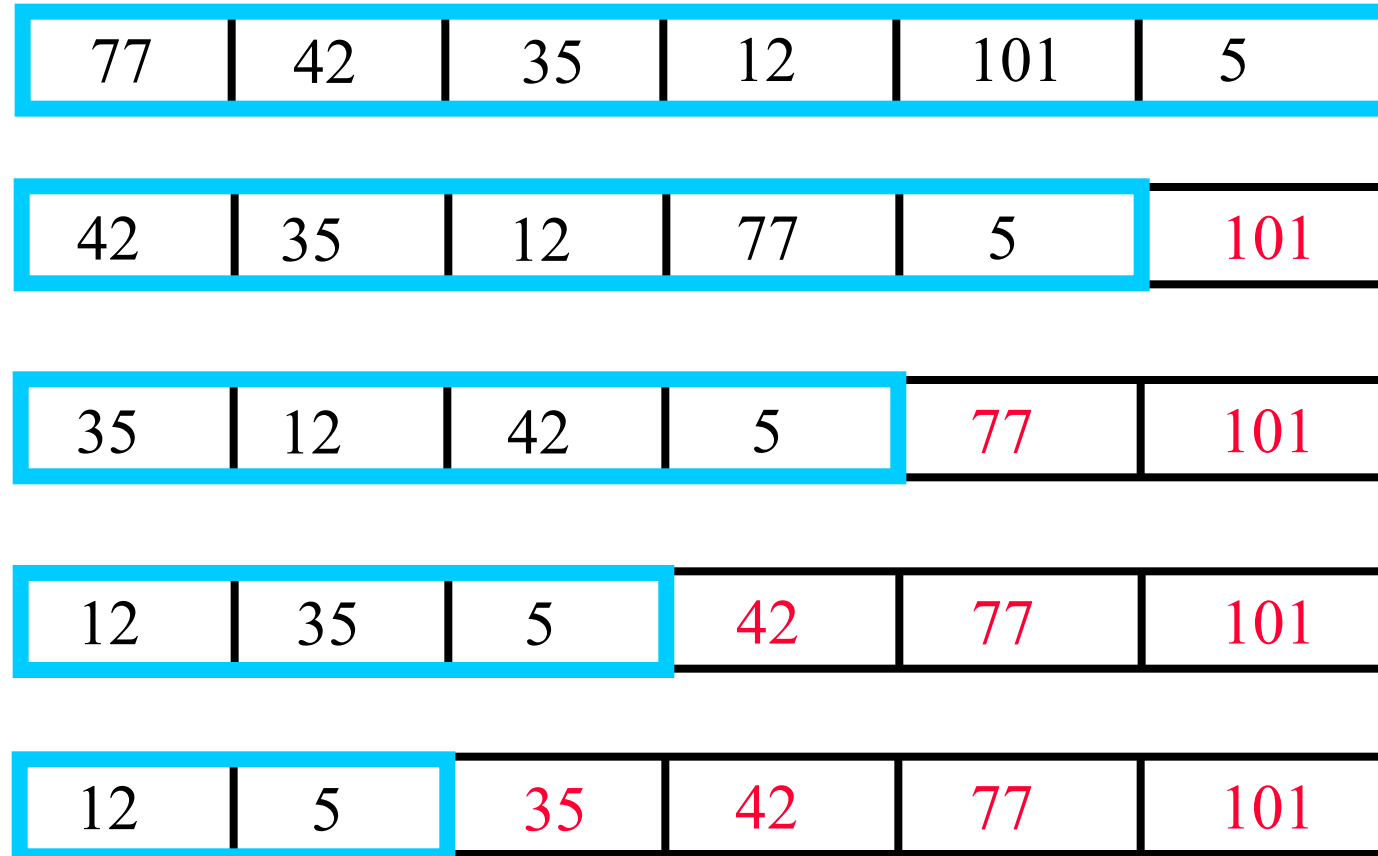
Repeat “Bubble Up” How Many Times?

- If we have N elements...
- And if each time we bubble an element, we place it in its correct location...
- Then we repeat the “bubble up” process $N - 1$ times.
- This guarantees we'll correctly place all N elements.

“Bubbling” All the Elements

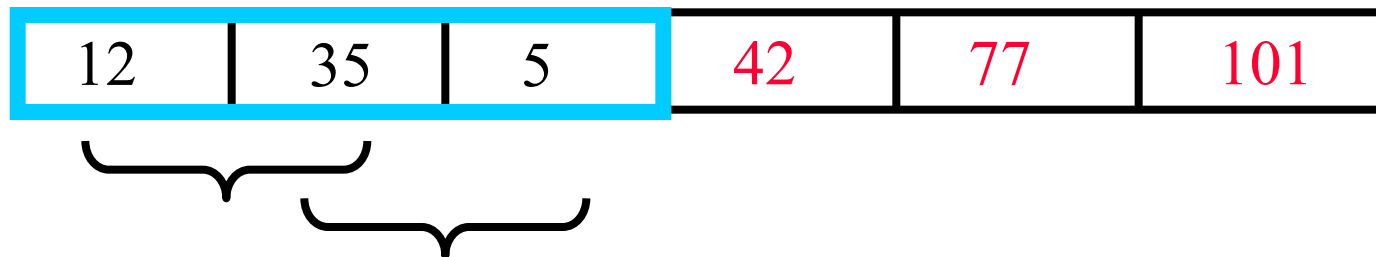


Reducing the Number of Comparisons



Reducing the Number of Comparisons

- On the i^{th} “bubble up”, we only need to do $N-i$ comparisons.
- For example:
- This is the 4th “bubble up”
- N is 6
- Thus we have 2 comparisons to do



Already Sorted Collections?

- What if the array A was already sorted?
- What if only a few elements were out of place and after a couple of “bubble ups,” the collection was sorted?
- How to detect this and stop early?

5	12	35	42	77	101
---	----	----	----	----	-----

Already Sorted Collections?

- We can use a **boolean variable** to determine if any swapping occurred during the “bubble up.”
- If no swapping occurred, then we know that the collection is already sorted!
- This boolean “flag” needs to be reset after each “bubble up.”

Putting It all Together

```
# Optimized Python program for implementation of Bubble Sort
def bubbleSort(arr):
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):
        swapped = False

        # Last i elements are already in place
        for j in range(0, n-i-1):

            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater
            # than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if (swapped == False):
            break
```

<https://www.geeksforgeeks.org/bubble-sort-algorithm/>

- There are n iterations and the running time of each iteration is $O(n)$
- Worst-case running time is $O(n^2)$

Practice 1

- [9, 4, 7, 2, 5]
- How many rounds to sort the array above?
- Write the array after **each bubble-up pass**.

[4,7,2,5,9]

[4,2,5,7,9]

[2,4,5,7,9]

No swaps, stop here

Outline

- Basic Sorting
 - Bubble Sort
 - **Selection Sort**
 - Counting Sort
- Linked List
 - Singly Linked List
 - Doubly Linked List
 - Link-Based Vs Array-based Sequences

Selection Sort – An example



Imagine you're in a library trying to **arrange books on a shelf in ascending order of their heights.**

To accomplish this, you decide to follow this method:

1. Start by scanning the entire shelf to find the **shortest** book.
2. Take the shortest book and place it at the **first** position.
3. Move to the next position and repeat the process: scan the remaining books, find the **next shortest** book, and place it in the next position.
4. Repeat until all books are in ascending order by height.



Selection Sort

- **Idea:**

- Find the smallest element in the array
- Exchange it with the element in the first position
- Find the second smallest element and exchange it with the element in the second position
- Continue until the array is sorted

Selection Sort - Example

8	4	6	9	2	3	1
---	---	---	---	---	---	---

1	4	6	9	2	3	8
---	---	---	---	---	---	---

1	2	6	9	4	3	8
---	---	---	---	---	---	---

1	2	3	9	4	6	8
---	---	---	---	---	---	---

1	2	3	4	9	6	8
---	---	---	---	---	---	---

1	2	3	4	6	9	8
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

1	2	3	4	6	8	9
---	---	---	---	---	---	---

Selection Sort - Algorithm

Alg.: SELECTION-SORT(A)

$n \leftarrow \text{length}[A]$

for $i \leftarrow 0$ to $n - 2$

 do $\text{smallest} \leftarrow i$

 for $j \leftarrow i + 1$ to $n - 1$

 do if $A[j] < A[\text{smallest}]$

 then $\text{smallest} \leftarrow j$

 exchange $A[i]$  $A[\text{smallest}]$

8	4	6	9	2	3	1
---	---	---	---	---	---	---

- There are $\mathbf{O}(n)$ iterations and each iteration finishes in $\mathbf{O}(n)$ time.
- The total running time is $\mathbf{O}(n^2)$

Recursive Selection Sort

- Redefine the iterative Selection Sort algorithm using **recursion**
- **recursive_sort(A, 0, n-1):**
 1. Find the smallest element in the array in $O(n)$ time
 2. Exchange it with the element in the first position
 3. We now have a **subproblem**:
 - sort the last $n - 1$ elements by calling **recursive_sort(A, 1, n-1)**

Analysis of Recursive Selection Sort

- Let $f(n)$ be the maximum running time of selection sort when the problem size is n .
- We know: $f(1) = O(1)$
- For $n \geq 2$, we have:
$$f(n) \leq O(n) + f(n - 1)$$
- where the term $O(n)$ captures the cost of Steps 1 and 2, and $f(n - 1)$ is the cost of Step 3.

Analysis of Recursive Selection Sort

- So it remains to solve the recurrence (c_1, c_2 are constants):
 - $f(1) = c_1$
 - $f(n) \leq c_2n + f(n-1)$
- Using the expansion method, we get:
- $f(n) \leq c_2n + f(n-1)$
 - $\leq c_2n + c_2(n-1) + f(n-2)$
 - $\leq c_2n + c_2(n-1) + c_2(n-2) + f(n-3)$
 - $\leq c_2n + c_2(n-1) + \dots + c_2 \cdot 2 + f(1)$
 - $\leq c_2n(n+1)/2 + c_1 = O(n^2)$.
- We now conclude that selection sort runs in $O(n^2)$ worst-case time.

Practice 2

- Run selection sort on:
 - [6, 1, 8, 3]
- Which index is swapped each round?

Round 1 → swap 6 & 1

Round 2 → swap 6 & 3

Round 3 → already correct

Quiz

- Which performs fewer swaps?
 - Bubble sort
 - Selection sort

Selection sort → at most n swaps.

Bubble sort → potentially MANY.

Swaps = memory writes = energy cost.

Outline

- Basic Sorting
 - Bubble Sort
 - Selection Sort
 - **Counting Sort**
- Linked List
 - Singly Linked List
 - Doubly Linked List
 - Link-Based Vs Array-based Sequences

A Simple Algorithm for Sorting?

Consider an array A of n integers. Every integer is in the range of $[1, U]$, where $U \geq n$.
Example: $[2, 3, 8, 7, 1, 2, 2, 2, 7, 3, 9, 8, 2, 1, 4, 2, 4, 6, 9, 2]$.

A simple and direct approach to make these numbers in order:

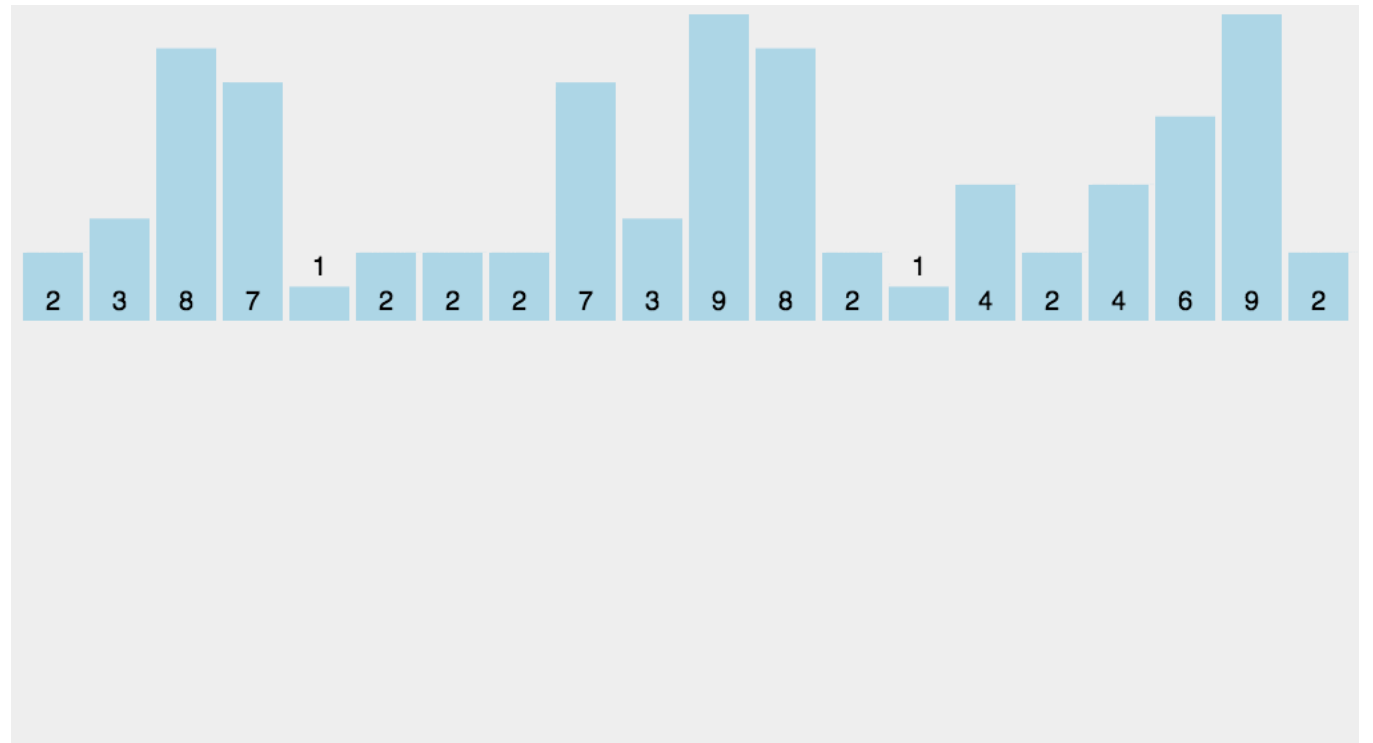
1. **Create a Count Array B** of length U . Initialize B by setting all its cells to 0.
2. **Count the Occurrences:** For every $i \in [0, n - 1]$, increase $B[A[i]]$ by 1.
3. **Generate the sorted order as follows** (set A to empty):
 - For $x = 1$ to U
 - while $B[x] > 0$ do
 - append integer x to A .
 - $B[x] = B[x] - 1$

Example

Consider a collection of integers: **[2,3,8,7,1,2,2,2,7,3,9,8,2,1,4,2,4,6,9,2]**.

A simple and direct approach to make these numbers in order:

1. **Create a Count Array**
2. **Count the Occurrences**
3. **Traverse and output**



A Simple Algorithm for Sorting?

The algorithm is called **counting sort**. It can handle a special case of the sorting problem where the integers come from a small domain.

Two main **properties**:

- **Non-Comparison-Based**: Relies on counting the frequency of elements and using array indices rather than comparisons.
- **Limited Applicability**: Only works for integers or discrete data that can be mapped to integers.

Analysis of Counting Sort

- Steps 1 and 3 take $O(U)$ time.
- Step 2 takes $O(n)$ time.
- Therefore, the overall running time of counting sort is $O(n + U) = O(U)$.
- For small $U = O(n)$ (e.g., $1000n$), the counting sort runs in $O(n)$ time.

Practice 3

- [2,1,3,2,0,1,2]
 - Step 1 — Create count array
 - Step 2 — Reconstruct sorted output

0 → 1

1 → 2

2 → 3

3 → 1

[0,1,1,2,2,2,3]

Practice 4

- Trace
 - [5,4,3,2]
 - [2,4,3,5]
 - [2,3,4,5]

Bubble or Selection?

Selection. Because the smallest is placed immediately.

Practice 5

- [8,3,6,2]
 - $\rightarrow$ [2,3,6,8]
 - $\rightarrow$ [3,6,8,2]
 - $\rightarrow$ [8,3,2,6]
-
- Which sequence will NOT happen in Bubble Sort?
 $\rightarrow$ [8,3,2,6], after first round, 8 will be at last position

Practice 6

- for i in range(n):
- for j in range(n):
- if $A[j] > A[j+1]$:
- swap

What is wrong?

- Out-of-bound access
- If this is bubble sort: inner loop should stop at $n-i-1$

Middle Exam

- Date: March 29th 2026
- Time: 15:00-17:00
- Location: Lecture Hall A

In-Class Quiz

- 2 in-class quiz test will be conducted for final grade.
- First time In March (likely the first week)
- Second time (TBD)
- Roughly 30 minutes each session

Slides

- Updated with answers

Practice 7

- Which algorithm performs BEST?
 - [1,2,3,5,4]

Bubble/Selection/Count?

Quiz

- **Q1.** What is the core idea shared by many sorting algorithms?
- A. Divide the array into random parts
- B. Maintain a sorted portion and an unsorted portion
- C. Always swap the first and last elements
- D. Use recursion

Comparison Among Sorting Algorithms

Sorting Algorithm	Time Complexity	Space Complexity	Stability
Insertion Sort	- Best: $O(n)$ (sorted) - Worst: $O(n^2)$	$O(1)$	Stable
Bubble Sort	- Best: $O(n)$ (sorted) - Worst: $O(n^2)$	$O(1)$	Stable
Selection Sort	- $O(n^2)$	$O(1)$	Not Stable
Counting Sort	- $O(n+U)$	$O(U)$	Stable

An algorithm is **stable** if numbers with the same value appear in the output array in the same order as they do in the input array.

Outline

- Basic Sorting
 - Bubble Sort
 - Selection Sort
 - Counting Sort
- **Linked List**
 - Singly Linked List
 - Doubly Linked List
 - Link-Based Vs Array-based Sequences

Linked List– An example

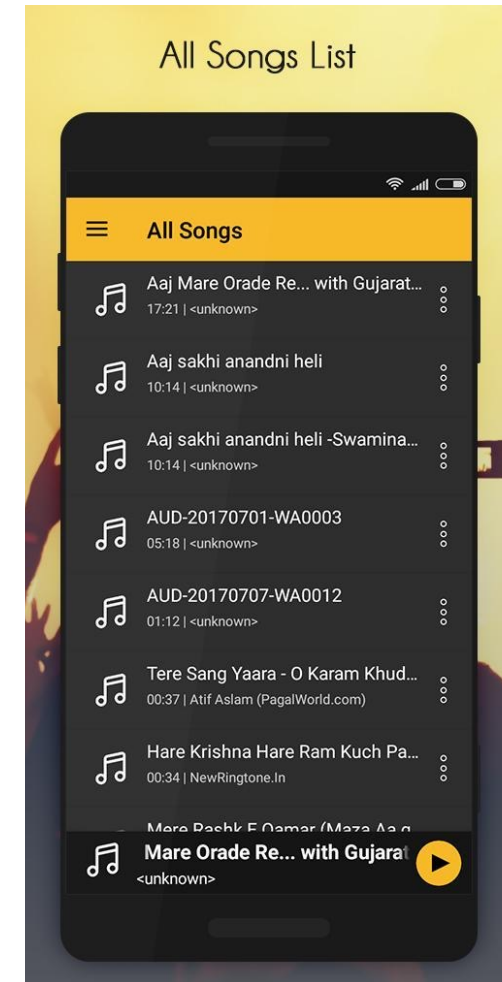


Imagine you are developing a music player application that allows users to create and manage playlists.

Users frequently add and remove songs from their playlists. Common operations include:

- Inserting a new song at a specific position
- Deleting an existing song from the list

Question: Can we use a dynamic array to manage the playlists? What are the advantages or disadvantages?



Introduction: Arrays vs. Linked Lists (1)

- **Array-based Sequences**

- Continuous memory block holding multiple elements.

- **Limitations**

- Dynamic arrays can have extra unused space.
- Inserting or deleting elements in the middle is costly.

Introduction: Arrays vs. Linked Lists (2)

- **Linked Lists:**
 - Alternative to arrays that avoids their limitations.
 - Uses nodes for each element, with references to neighbors to maintain order.
- **Limitations:**
 - Inefficient to access elements by index.
 - Can't determine a node's position in the list by examining the node only.

Outline

- Basic Sorting
 - Bubble Sort
 - Selection Sort
 - Counting Sort
- Linked List
 - **Singly Linked List**
 - Doubly Linked List
 - Link-Based Vs Array-based Sequences

Singly Linked Lists (0)

```
class ListNode :  
    def __init__( self, data ) :  
        self.data = data
```

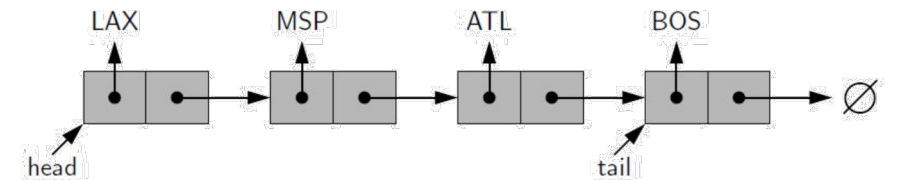
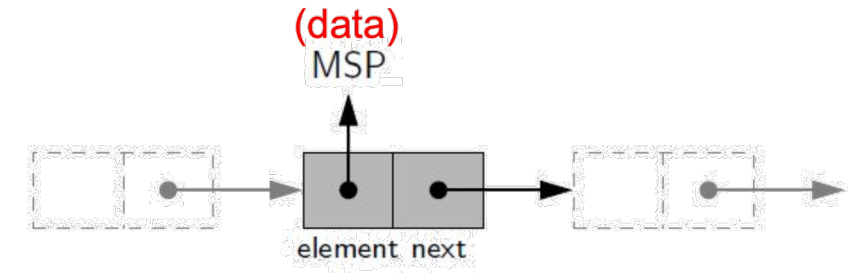
```
a = ListNode( 11 )  
b = ListNode( 52 )  
c = ListNode( 18 )
```

```
class ListNode :  
    def __init__( self, data ) :  
        self.data = data  
        self.next = None
```

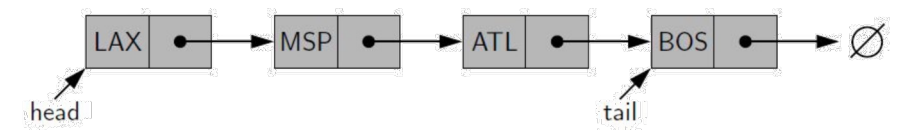
```
a.next = b  
b.next = c
```

Singly Linked Lists (1)

- A singly linked list is a sequence of **nodes**
 - **Node Contents:** Each node holds an **element** and a **reference** to the next node.
 - **Non-Continuous Storage:** Nodes are not required to be in consecutive memory addresses
 - **Head and Tail:** The first node is the head; the last node is the tail.
 - **Traversal:** Start at the head and follow next references to reach the tail, known as **link hopping** or **pointer hopping**.



Singly linked list for storing airport codes



Simplified view

Singly Linked Lists (2)

Minimally, the linked list instance should keep:

- **Head Reference:** Needed to locate the first node and access the entire list.
- **Tail Reference:** Optional but useful to quickly access the last node without traversing the list.
- **Size Count:** Optional but helpful to track the number of nodes without having to traverse the list each time.

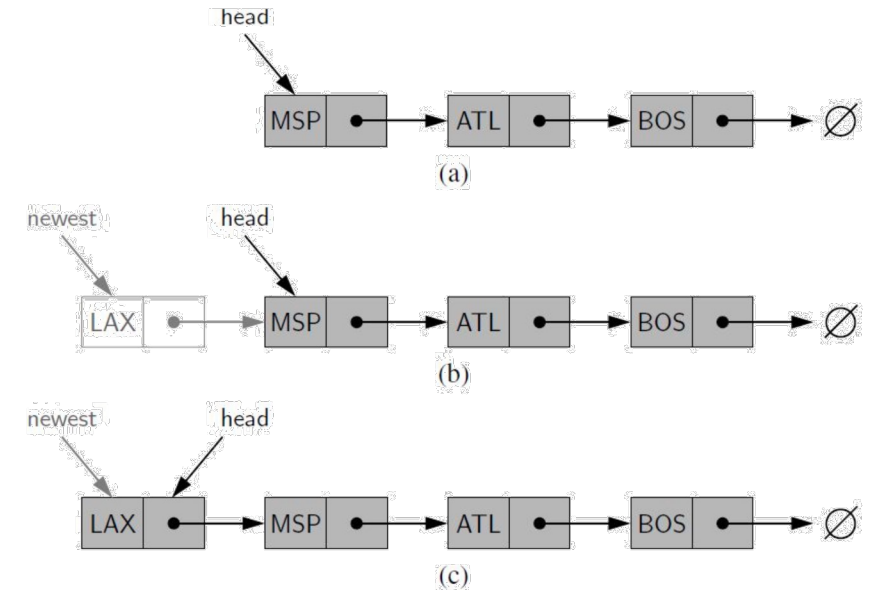
Traversing a linked list and Searching for a node

```
1 def traversal( head ):  
2     curNode = head  
3     while curNode is not None :  
4         print curNode.data  
5         curNode = curNode.next
```

```
1 def unorderedSearch( head, target ):  
2     curNode = head  
3     while curNode is not None and curNode.data != target :  
4         curNode= curNode.next  
5     return curNode is not None
```

Inserting an Element at the Head of a Singly Linked List

- An important property:
 - the space is proportional to the current number of elements.
 - we can easily insert an element at the head of the list.
- Steps for inserting an element:
 - Create a new node.
 - Set its element to the new element.
 - Set its next link to refer to the current head.
 - Set the list's head to point to the new node.



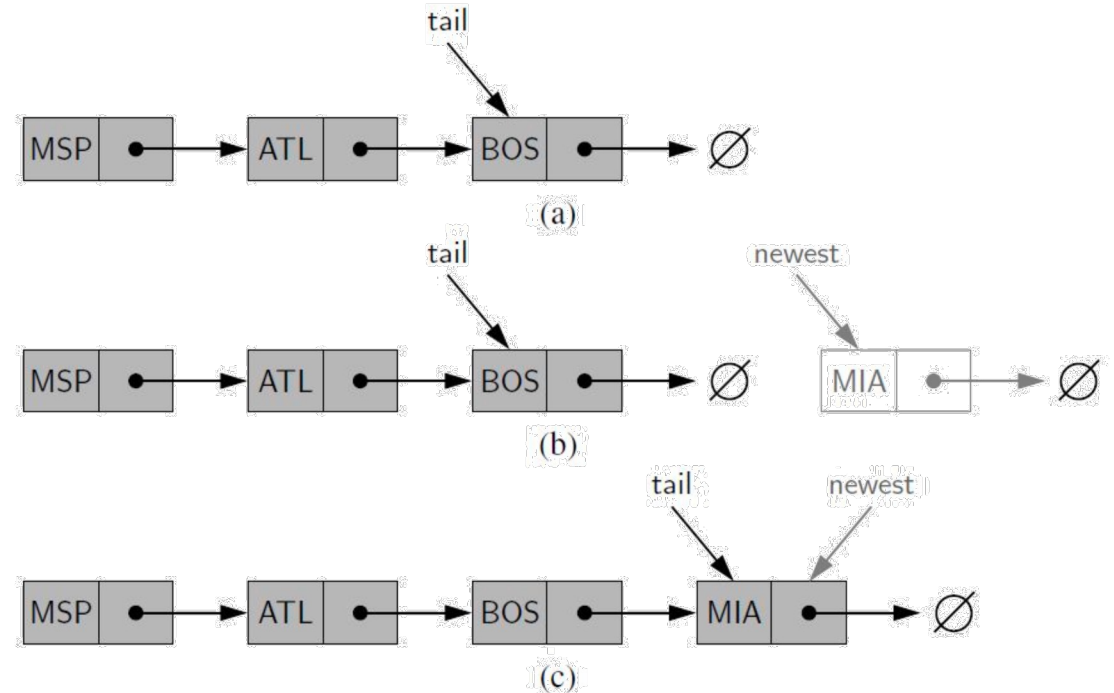
Algorithm add_first(L,e):

```
newest = Node(e) {create new node instance storing reference to element e}
newest.next = L.head {set new node's next to reference the old head node}
L.head = newest {set variable head to reference the new node}
L.size = L.size + 1 {increment the node count}
```

Inserting an Element at the Tail of a Singly Linked List

Steps:

- Create a new node.
- Assign its next reference to None.
- Set the next reference of the tail to point to this new node.
- Update the tail reference itself to this new node.

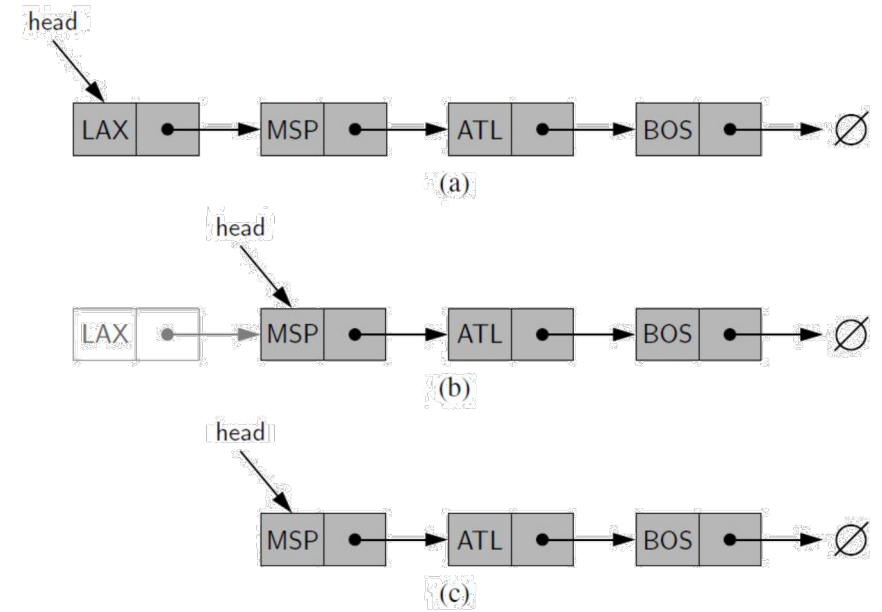


Algorithm add_last(L, e):

```
newest = Node(e) {create new node instance storing reference to element e}
newest.next = None {set new node's next to reference the None object}
L.tail.next = newest {make old tail node point to new node}
L.tail = newest {set variable tail to reference the new node}
L.size = L.size + 1 {increment the node count}
```

Removing an Element from a Singly Linked List

- **Removing from Head:** Easy, just reverse the insert-at-head process.
- **Removing Last Node:** Difficult because you need access to the node before the last one.
- **Accessing Preceding Node:** Requires starting at the head and searching through the list.

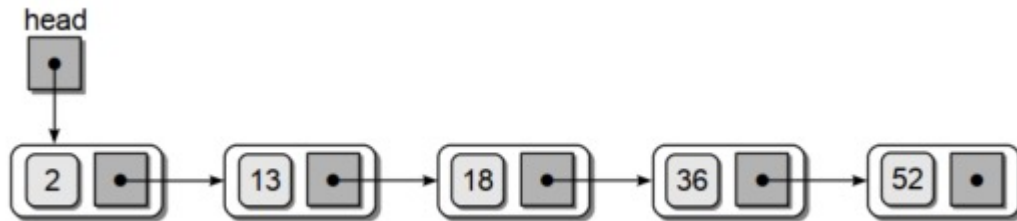


```
Algorithm remove_first(L):  
    if L.head is None then  
        Indicate an error: the list is empty.  
    L.head = L.head.next  
    L.size = L.size - 1
```

{make head point to next node (or None)}
{decrement the node count}

Sorted Linked List

- Items in a linked list can be sorted in ascending or descending order



Searching a sorted linked list

```
1 def sortedSearch( head, target ) :  
2     curNode = head  
3     while curNode is not None and curNode.data < target :  
4         if curNode.data == target :  
5             return True  
6         else :  
7             curNode = node.next  
8     return False
```

Inserting a value into a sorted list

```
1 # Given the head pointer, insert a value into a sorted linked list.  
2 # Find the insertion point for the new value.  
3 predNode = None  
4 curNode = head  
5 while curNode is not None and value > curNode.data :  
6     predNode = curNode  
7     curNode = curNode.next  
8  
9 # Create the new node for the new value.  
10 newNode = ListNode( value )  
11 newNode.next = curNode  
12 # Link the new node into the list.  
13 if curNode is head :  
14     head = newNode  
15 else :  
16     predNode.next = newNode
```

Singly Linked List Practice 1

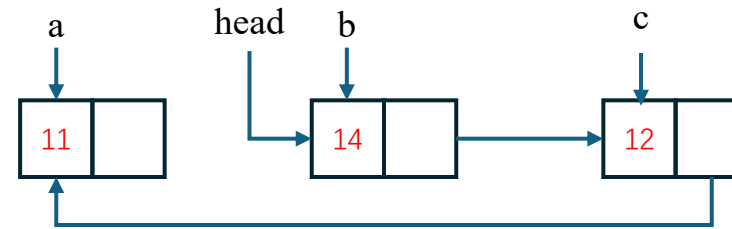
Accordingly to code below:

- Draw the memory structure (boxes + arrows).
- Label **head**, **a**, **b**, **c**.

```
class Node:
    def __init__(self, v):
        self.val = v
        self.next = None
```

```
a = Node(11)
b = Node(14)
c = Node(12)
```

```
c.next = a
b.next = c
head = b
```



Singly Linked List Practice 2

- Assume a singly linked list of size n

Operation	Time Complexity	
Insert at head	?	$O(1)$
Insert at tail (no tail ref)	?	$O(n)$
Search by value	?	$O(n)$
Access index i	?	$O(n)$

Singly Linked List Practice 3

- You are given a reference to a node cur

Task: Insert value x immediately after cur

```
new = Node(x)
# fill in the missing lines

new.next = cur.next
cur.next = new
```

Task: What happens if the order is reversed?

```
cur.next = new
new.next = cur.next
```

The new node next will be itself

Singly Linked List Practice 4

- Given

```
head = head.next
```

- Question:

1. Is the old head deleted? No
2. What happens in memory? But no accessible anymore

Singly Linked List Practice 5

- You are given a reference to a node x (not the head)
- **Question:** Can you delete it?

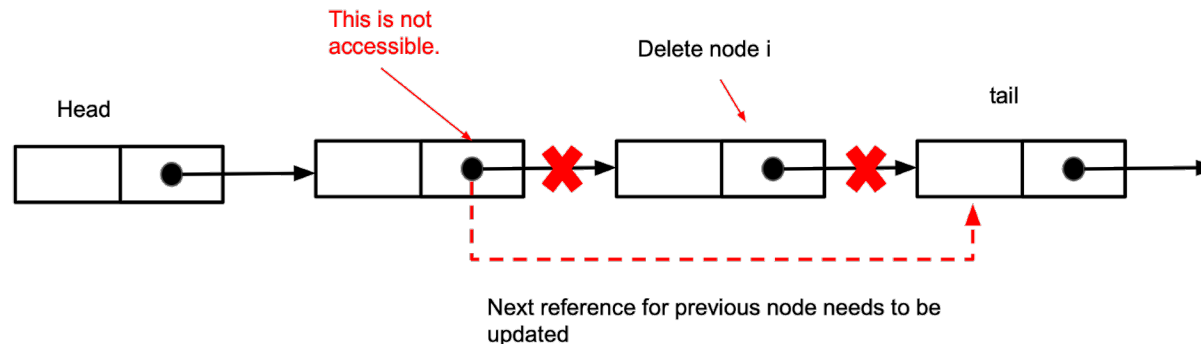
No, as we can't access previous node in singly linked list.

Outline

- Basic Sorting
 - Bubble Sort
 - Selection Sort
 - Counting Sort
- Linked List
 - Singly Linked List
 - **Doubly Linked List**
 - Link-Based Vs Array-based Sequences

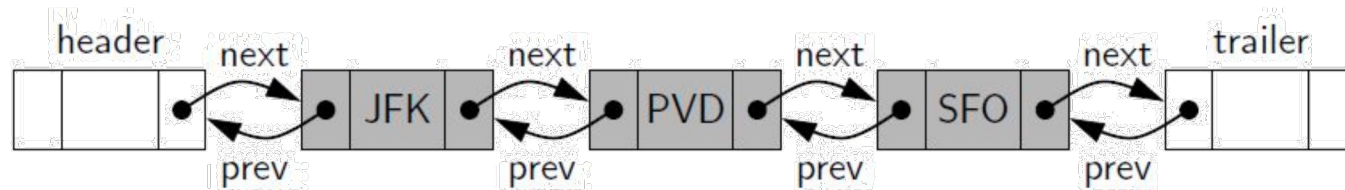
Doubly Linked List - Motivation

- **Singly Linked List Limitation:** Each node only points to the next one.
- Challenges:
 - Hard to efficiently remove the last node.
 - More generally, difficult to remove any node if you only have a reference to it, because you can't easily find the previous node to update its link.



Doubly Linked List (1)

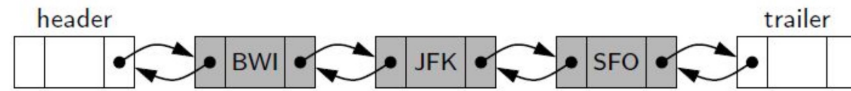
- **Structure:** Each node has links to both the next and previous nodes.
- **Advantages:**
 - Quick insertions and deletions anywhere in the list ($O(1)$ time).
- **Sentinel Nodes:**
 - Use special "header" and "trailer" nodes at the start and end.
 - These dummy nodes make handling edge cases easier.
 - In an empty list, the header points to the trailer, and the trailer points back to the header.



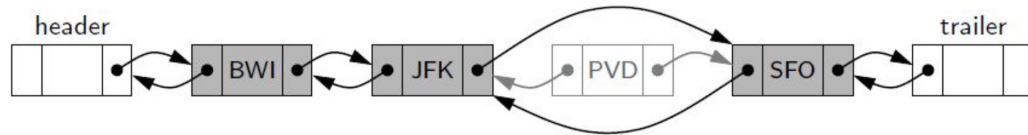
Doubly Linked List (2)

- **Sentinel Nodes:**
 - Simplify operations by keeping header and trailer fixed.
- **Unified Operations:**
 - Treat insertions and deletions as changes between existing nodes.

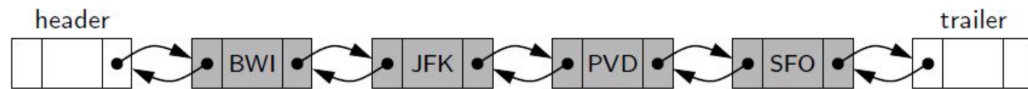
Inserting with a Doubly Linked List



(a)



(b)



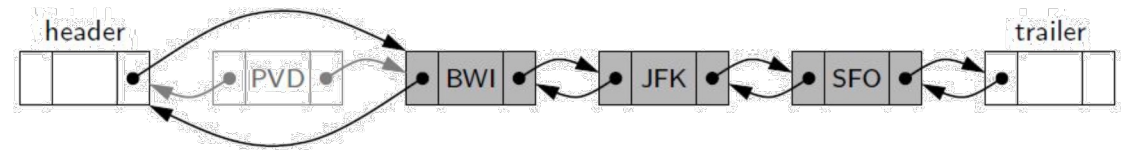
(c)

Case I: Inserting at an arbitrary position in a non-empty list.

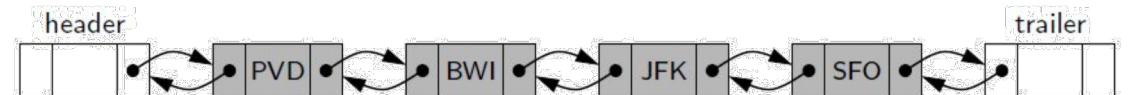
Case II: Inserting at the beginning of the list



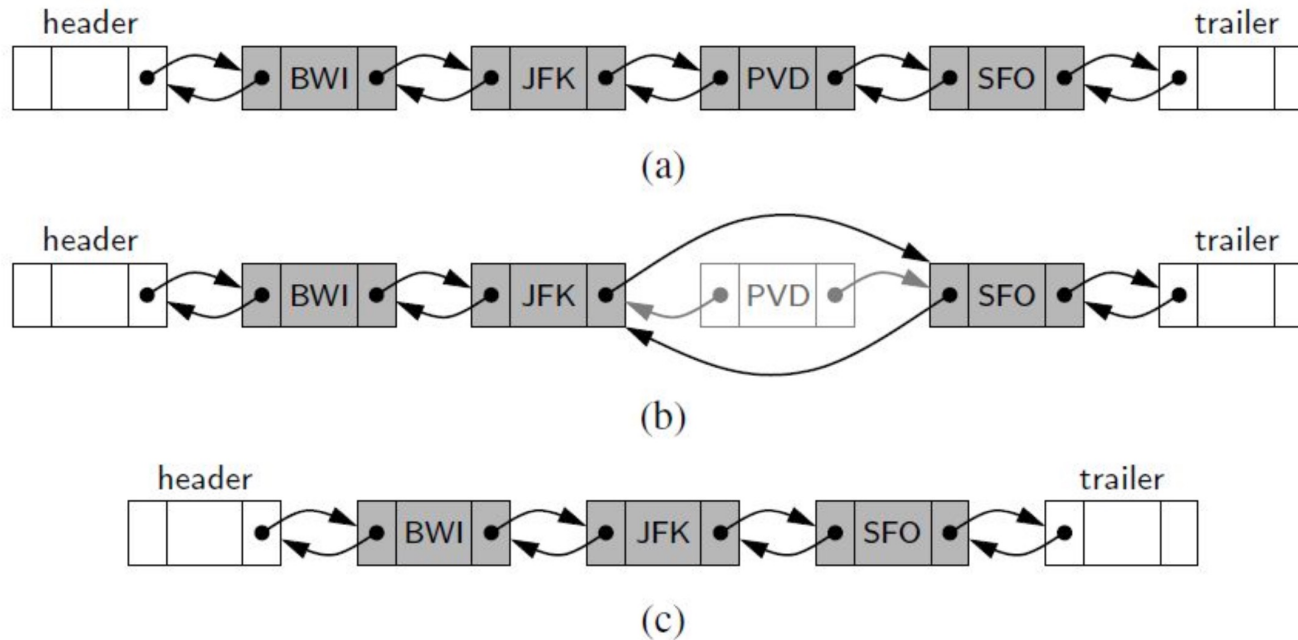
(a)



(b)



Deleting a Node in a Doubly Linked List



Due to the use of sentinels, the same implementation can be used for deleting the first or the last node.

Removing a node at an arbitrary location

Basic Implementation of a Doubly Linked List (1)

```
1 class _DoublyLinkedBase:
2     '''A base class providing a doubly linked list representation.'''
3
4     #-----
5     class _Node:
6         '''Lightweight, nonpublic class for storing a doubly linked node.'''
7         __slots__ = '_element', '_prev', '_next'      # streamline memory
8
9         def __init__(self, element, prev, next): # initialize node's field
10             self._element = element             # element to be stored
11             self._prev = prev                    # Previous node reference
12             self._next = next                    # next node reference
13     #-----
14     def __init__(self):
15         '''Create an empty list.'''
16         self._header = self._Node(None, None, None)
17         self._trailer = self._Node(None, None, None)
18         self._header._next = self._trailer      # trailer is after header
19         self._trailer._prev = self._header      # header is before trailer
20         self._size = 0                          # Number of elements
21
22     def __len__(self):
23         '''Return the number of elements in the list.'''
24         return self._size
25
26     def is_empty(self):
27         '''Return True if list is empty.'''
28         return self._size == 0
```

Base Class declaration

We maintain two
references: **`_prev`**
& **`_next`**

Basic Implementation of a Doubly Linked List (1)

```
30 def _insert_between(self, e, predecessor, successor):
31     '''Add element e between two existing nodes and return new node.'''
32     newest = self._Node(e, predecessor, successor) # linked to neighbors
33     predecessor._next = newest
34     successor._prev = newest
35     self._size += 1
36     return newest
37
38 def _delete_node(self, node):
39     '''Delete nonsentinel node from the list and return its element.'''
40     predecessor = node._prev
41     successor = node._next
42     predecessor._next = successor
43     successor._prev = predecessor
44     self._size -= 1 # record deleted element
45     element = node._element
46     node._prev = node._next = node._element = None # deprecate node
47     return element # return deleted element
```

Doubly Linked List Practice 1

- Compare Lists

Operation	Singly	Doubly	
Delete given node	?	?	$O(n)$ vs $O(1)$
Traverse backward	?	?	impossible vs $O(n)$ backward
Memory cost	?	?	lower vs higher

Doubly Linked List Practice 2

- Fill in the missing pointers

- Initial list:

$H \leftrightarrow A \leftrightarrow C \leftrightarrow T$

- insert node B **between A and C**

B.prev = ____

B.next = ____

____.next = ____

____.prev = ____

B.prev = A

B.next = C

A.next = B

C.prev = B

Doubly Linked List Practice 3

- You have:

`A <-> C`

- You want to insert B between them. Consider this code:

```
A.next = B
B.prev = A
B.next = C
C.prev = B
```

Is there any moment when the list becomes unreachable / corrupted?

*This order is safe if you already have references to A, B, C as variables.
But in many real cases you only have A and A.next (which is C). If you overwrite
A.next too early without saving C, you lose the reference to C.*

Doubly Linked List Practice 4

- List

H <-> A <-> B <-> C <-> T

- You are given variable b referencing node B.

A.next = B

B.prev = A

B.next = C

C.prev = B

Task: Delete B in O(1), no traversal.

b.prev.next = b.next

b.next.prev = b.prev

Doubly Linked List Practice 5

- Given

... <-> A <-> B <-> ...

- You are given references to A and B and know they are adjacent with $A.next == B$.
- **Task:** Swap them so it becomes:

... <-> B <-> A <-> ...

Let $P = A.prev$, $N = B.next$

$P.next = B$

$B.prev = P$

$B.next = A$

$A.prev = B$

$A.next = N$

$N.prev = A$

Doubly Linked List Practice 6

- Two lists (each with sentinels):

- List1: `H1 <-> A <-> B <-> T1`

`Let B = A.next.`

`A.next = X`
`X.prev = A`

- List2: `H2 <-> X <-> Y <-> T2`

`Y.next = B`
`B.prev = Y`

- **Task:** Splice the entire List2 (X..Y) **after node A** in List1, producing:

`H1 <-> A <-> X <-> Y <-> B <-> T1`

Assume you have references: A, B (A.next), X (H2.next), Y (T2.prev).

Link-Based Vs Array-Based Sequences

Feature	Array	Linked Lists
Access Time	$O(1)$ for accessing elements via index	$O(1)$ if knowing the reference to the node
Insertion/Deletion	$O(n)$, requires shifting elements	$O(1)$ at any position
Storage	Continuous	Consist of small blocks, not necessarily continuous
Memory Usage	$O(n)$ space; more memory-efficient, stores only elements	$O(n)$ space; less memory-efficient, stores element and pointers
Performance Consistency	May have variable performance due to resizing	Consistent performance, predictable time bounds
Use Case Efficiency	Better for scenarios requiring frequent access	Better for scenarios with frequent insertions/deletions